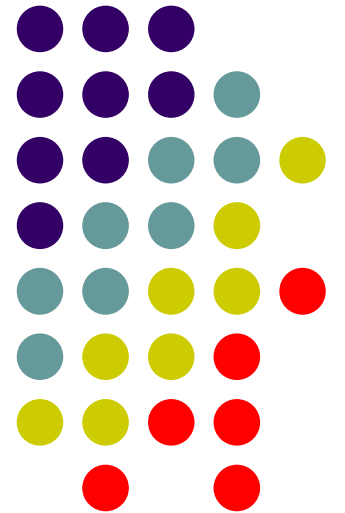
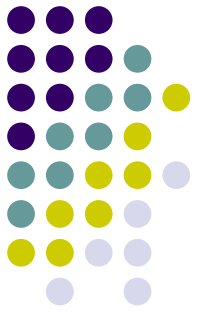


Ch3 Linux shell编程基础



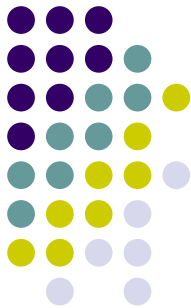


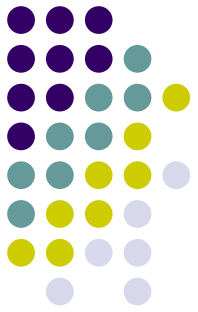
本章内容

- Shell概述
- Shell变量
- Shell符号
- Shell脚本
- 流程控制
- 函数
- 正则表达式

Part 1 Shell概述

- Shell类型
- Shell功能
- Shell用途

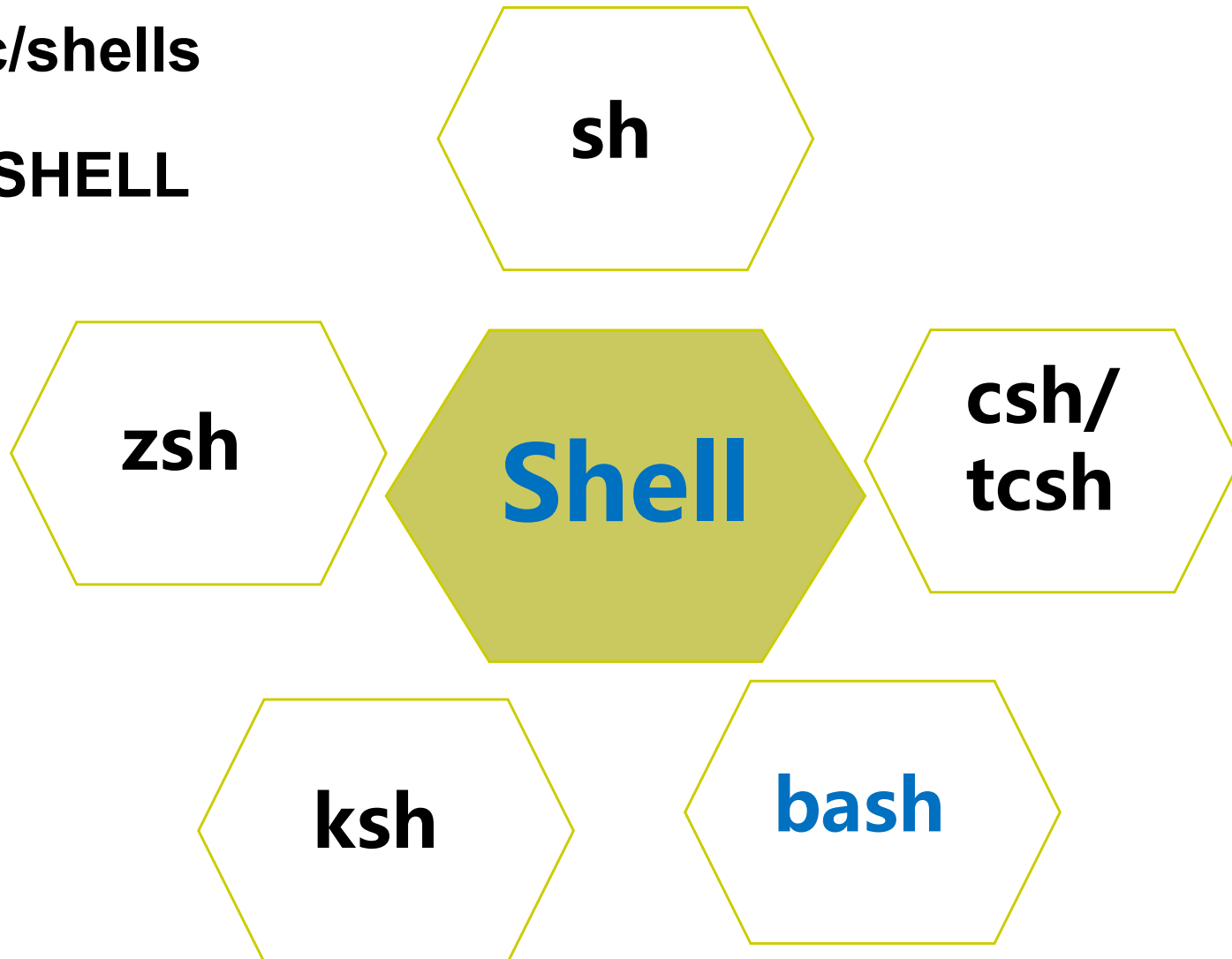


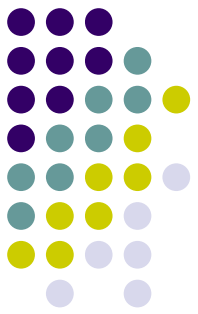


1.1 Shell类型

cat /etc/shells

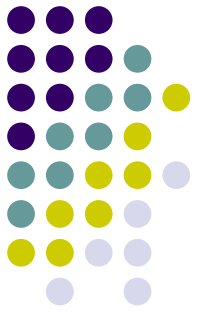
echo \$SHELL





1.2 Shell功能

- 命令解释器：连接用户和内核，用户应用程序
- 文件名替换，命令替换，变量替换
- 历史替换，别名替换
- 流程控制的内部命令



1.2 Shell功能

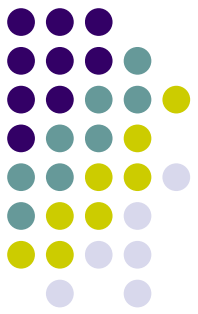
- 历史替换

查看历史表：内部命令history

文件\$HOME/.bash_history

- 使用历史记录

- 上下箭头键
- !!引用上一命令
- !str 以str开头的最近用过的命令



1.2 Shell功能

- 别名

alias

查看别名：内部命令alias

unalias

设置别名：alias mm=' mv -i '

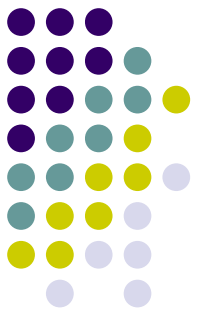
alias

unalias

取消别名：unalias n

unalias

```
[root@node01 ~]# alias
alias cp='cp -i'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l.='ls -d .* --color=auto'
alias ll='ls -l --color=auto'
alias ls='ls --color=auto'
alias mv='mv -i'
alias rm='rm -i'
```



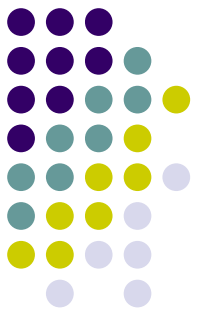
1.3 Shell用处

- **交互方式下**：熟悉shell的替换机制、转义机制，掌握循环等流程控制，可以编写复合命令
- **非交互方式**：编写shell脚本程序，把一系列的操作，写成一个脚本文件，批量处理

Part 2 Shell变量



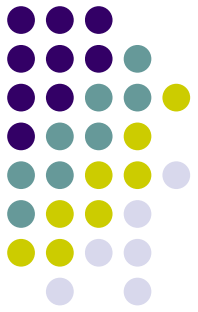
- 变量使用
- 本地变量
- 环境变量
- 位置变量
- 特殊变量



2.1 变量使用

- 变量命名

- 首个字符不能以数字开头
- 其余字符可以是字母，数字，下划线
- 中间不能有空格，可以使用下划线 _。
- 不能使用标点符号。
- 不能使用bash里的关键字（可用help命令查看保留关键字）



2.1 变量使用

不用定义类型

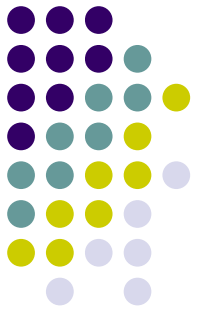
- 变量赋值

- 使用格式: **NAME=value**

- 使用举例:

addr=20.1.1.254

_school="qust"



2.1 变量使用

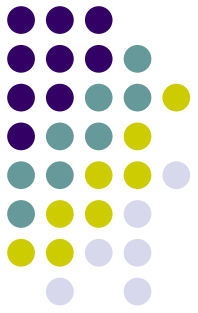
- 变量引用

- 使用格式: $\$NAME$ 或 $\${NAME}$ *后者使用.*

- 使用举例:

$\$addr$

$\${_school}.edu.cn$



2.1 变量使用

- 变量删除

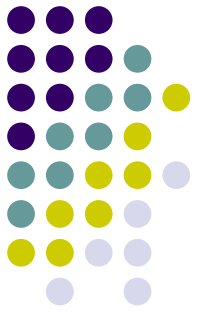
- 使用格式: unset NAME

unset 删除
变量

- 变量被删除后不能再次使用, 不能删除只读变量

- 使用举例:

unset addr

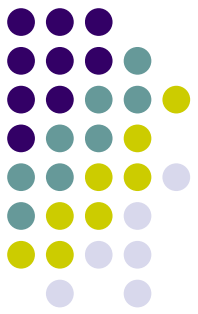


2.1 变量使用

- 变量输入命令read
 - 从标准输入读入一行内容赋值给变量
 - 使用格式: **read name**
 - 参数: -p 设置提示信息
 - 使用举例:

`read -p "input your name" NAME`

提示



2.1 变量使用

- 变量输出命令echo

⇔ printf

- 使用格式: `echo arg1 arg2 arg3 ...`

- 使用举例:

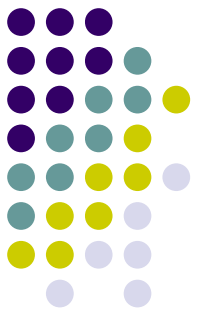
```
echo "Hello world"
```

```
echo "\"Hello world\""
```

```
echo "$name Hello world"
```

```
echo -e "Hello! \n"
```

```
echo -e "Hello! \c"
```



2.1 变量使用

- 变量输出命令printf

- 使用格式: `printf format-string [arguments...]`

- 格式字符:

- %s 输出一个字符串

- %d 整型输出

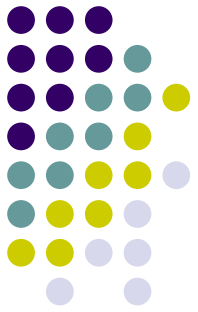
- %c 输出一个字符

- %f 输出实数, 以小数形式输出

- 使用举例:

- `printf "Hello, Shell\n"`

- `printf "%d %s\n" 1 "abc"`

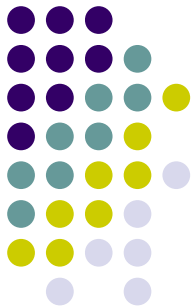


2.1 变量使用

- 字符串

字符串常见操作表达式

字符变量	含义
<code>\${#var}</code>	返回字符串变量var的长度
<code>\${var:m}</code>	返回 <code>\${var}</code> 中从第m个字符之后的所有字符
<code>\${var:m:len}</code>	返回 <code>\${var}</code> 中从第m个字符之后，长度为len的子串
<code>\${var#pattern}</code>	删除 <code>\${var}</code> 中开头部分与pattern匹配的子串
<code>\${var%pattern}</code>	删除 <code>\${var}</code> 中结尾部分与pattern匹配的子串

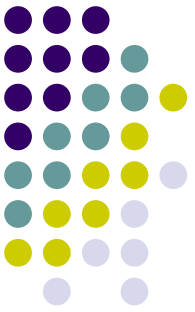


2.1 变量使用

- 字符串

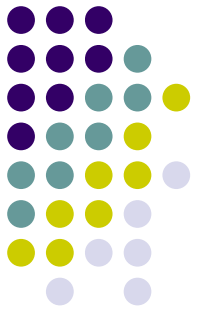
字符串常见操作表达式

字符变量	含义
<code>\${var/old/new}</code>	用new替换 <code>\${var}</code> 中第一次出现的old子串
<code>\${var//old/new}</code>	用new替换 <code>\${var}</code> 中所有的old子串
<code>\${var/#old/new}</code>	用new替换 <code>\${var}</code> 中开头与old匹配的子串
<code>\${var%old/new}</code>	用new替换 <code>\${var}</code> 中结尾与old匹配的子串



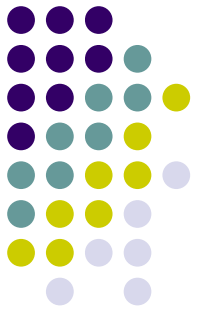
变量使用举例1

```
echo "===examples for testing read==="  
echo -e "What is your name?\c"  
read NAME  
echo "Hello $NAME"  
echo  
echo -n "Where are you work?"  
read  
echo "I guess $REPLY keeps you busy!"  
read -p "Enter your job title:"  
echo "I thought you might be an $REPLY"
```



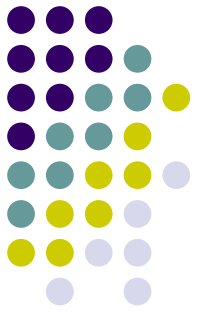
变量使用举例2

```
var="Hello shell,this is a testing of string"  
echo "The string is:  ${var}"  
echo "The sting length is:  ${#var}"  
echo "The No. 5 character is:  ${var:4:1}"  
echo "From No.2 character is:  ${var:1}"  
echo "Delete 'H' is :  ${var#H}"  
echo "Replace the first 'l' with 'L' is :  ${var/l/L}"  
echo "Replace all 'l' with 'L' is :  ${var//l/L}"
```

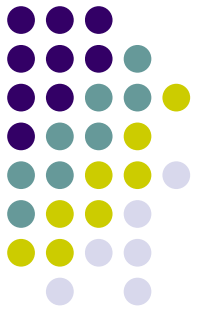
2.1 变量使用

- 变量类型
 - 本地变量（局部变量）
 - 环境变量
 - 位置变量
 - 特殊变量



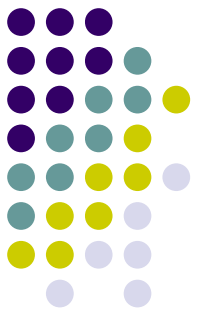
2.2 本地变量

在脚本或命令中定义，仅在当前shell实例中有效，其他shell启动的程序不能访问局部变量。



2.3 环境变量

- 环境变量定义
 - NAME=value (定义)
 - export NAME (导出, 非常重要)
 - echo \$NAME (使用)
 - unset NAME (清除)
 - env命令查看所有环境变量



2.3 环境变量

- 常见环境变量

- PATH

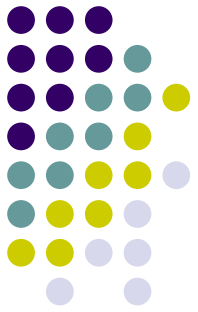
用于帮助Shell找到用户所输入的命令。

- PWD和OLDPWD

PWD记录当前系统目录； OLDPWD记录旧有的工作目录。

- HOME

HOME记录当前用户的根目录，用于保存用户自己的文件。



2.3 环境变量

- 常见环境变量

- SHELL

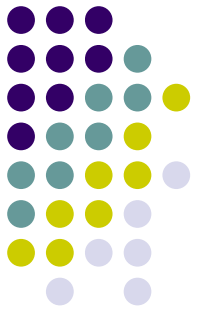
SHELL变量保存Shell的值，默认的Shell值/bin/bash。

- USER和UID

USER和UID是保存已登录用户的名字和ID。

- PS1和PS2

PS1和PS2称为提示符变量，用于设置提示符格式



2.3 环境变量

- 环境变量配置文件

- 系统级：对所用用户有效

/etc/profile 系统级配置文件

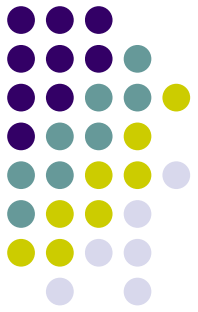
/etc/profile.d

/etc/bashrc

- 用户级：对当前用户有效

~/.bash_profile

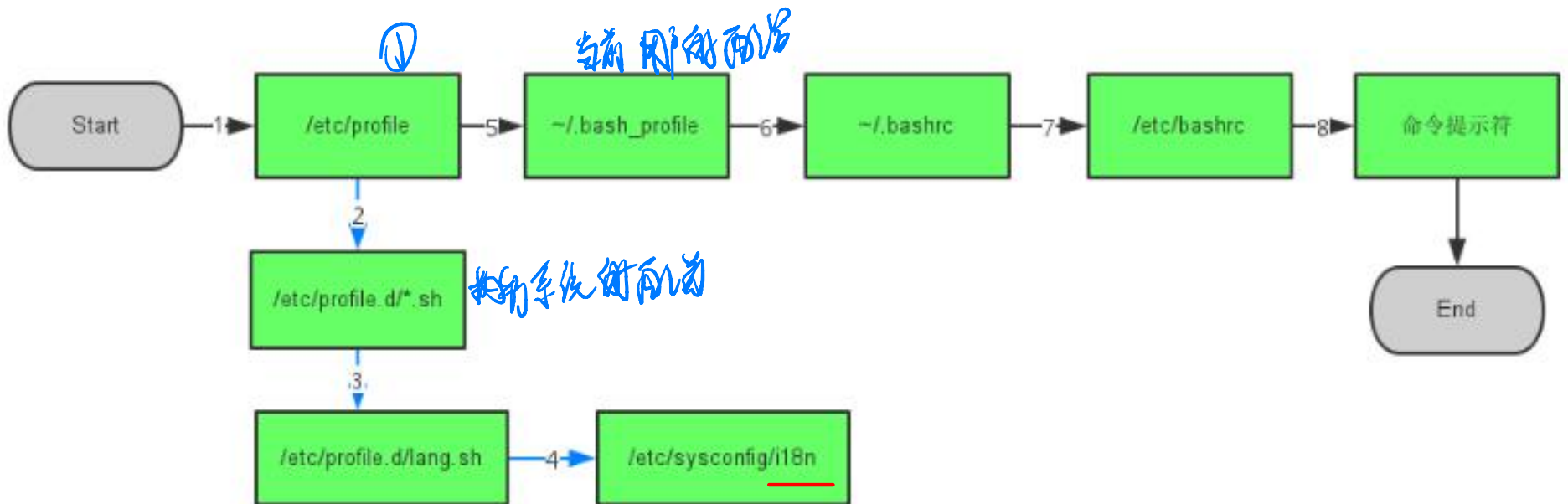
~/.bashrc

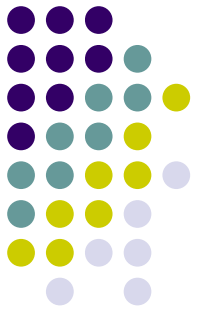


2.3 环境变量

- 环境变量配置文件

- 加载顺序——登录式用户



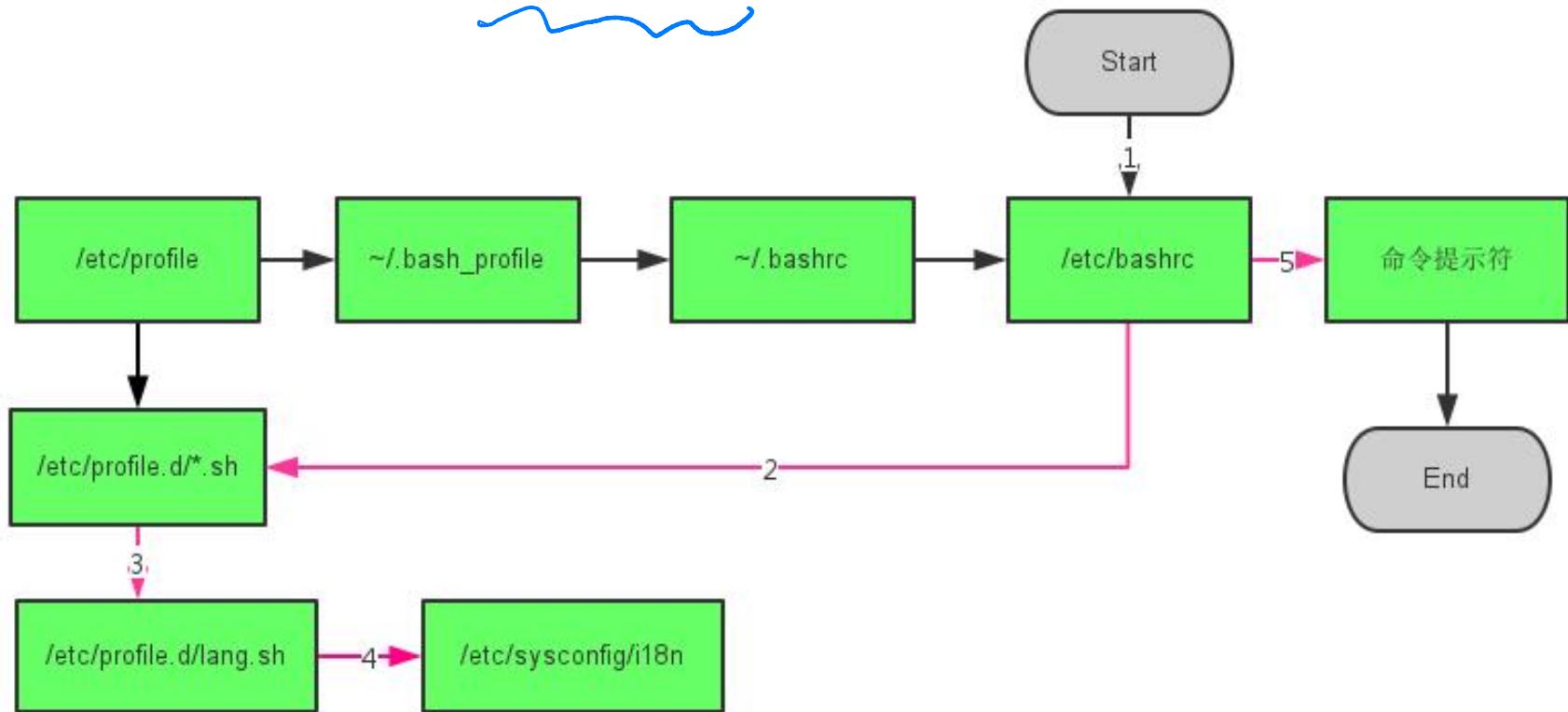


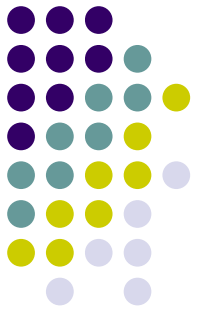
2.3 环境变量

- 环境变量配置文件

su 切换登录的用户

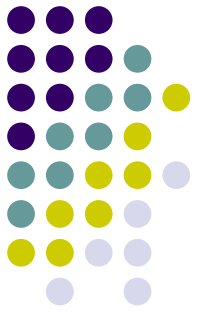
- 加载顺序——非登录式用户





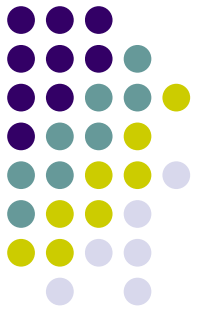
2.3 环境变量

- 环境变量配置文件/etc/profile文件
 - USER 变量
 - LOGNAME 变量
 - MAIL 变量
 - PATH 变量
 - HOSTNAME 变量
 - HISTSIZE 变量
 - umask
 - 遍历调用 /etc/profile.d/*.sh 文件



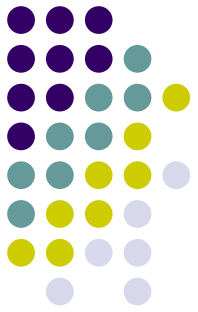
2.3 环境变量

- 环境变量配置文件~/.bash_profile文件
 - 调用 ~/.bashrc 文件
 - 在PATH变量后加入 :\$HOME/bin 目录



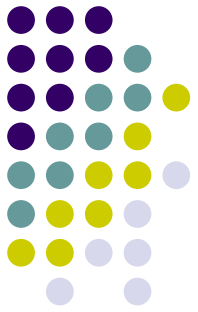
2.3 环境变量

- 环境变量配置文件~/.bashrc文件
 - 定义别名
 - 调用 /etc/bashrc 文件



2.3 环境变量

- 环境变量配置文件/etc/bashrc 文件
 - PS1 变量
 - PATH 变量（非登录用户）
 - umask（非登录用户）
 - 遍历调用 /etc/profile.d/*.sh 文件（非登录用户）



2.3 环境变量

- 配置文件修改生效

- 注销，重新登录

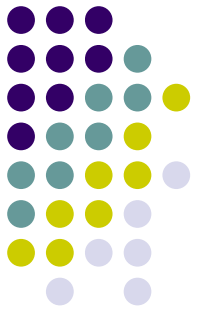
- source命令



• 配置文件

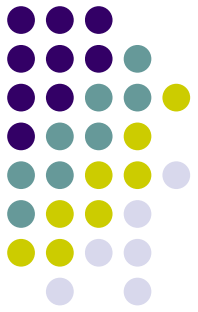
source /etc/profile

./etc/profile



2.4 位置变量

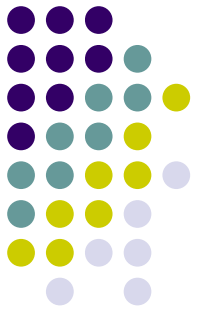
- 位置变量用来向Shell脚本传递参数
- 它是以单个整数来命名的：\$n
- \$n表示第n个位置参数
- 注意：\$符号不能少
- 例：\$0, \$1, \${10}



2.5 特殊变量

- **\$#** : 传递到脚本的参数个数。
- **\$***和**\$@** : 传递到脚本的所有参数。
- **\$?** : 命令退出状态: 0无错误, 非0错误。
- **\$\$** : 表示本程序的进程PID。
- **#!** : 上一个后台进程的PID

\$? 当前命令的执行状态



变量使用举例3

echo "Hello,\$USER,the output of this script are as follows:"

echo "The script name is: \$0"

echo "The first param of this script is: \$1"

echo "All the params you input are:\$@"

echo "All the params you input are:\$*"

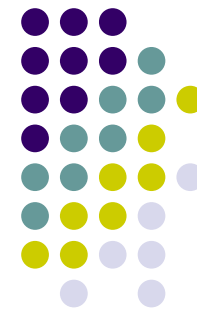
echo "The number of the params you input are:\$#"

echo "The process ID of the params you input are:\$\$"

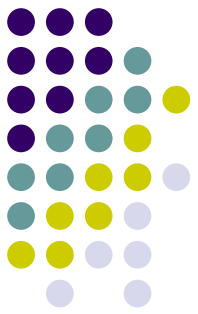
echo "The exit status of this script is:\$?"



Part 3 Shell符号



- 引号
- 通配符
- 连接符
- 其他符号



3.1 引号

单引号屏蔽\$、\、`
双引号不会屏蔽

- 单引号：它可以屏蔽Shell中元字符的功能

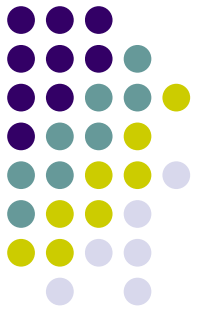
```
[root@node01 ~]# echo 'hello $PATH'
hello $PATH
```

- 双引号：屏蔽，但不会屏蔽\$、\、`符号

```
[root@node01 ~]# echo "$NUM"
10
```

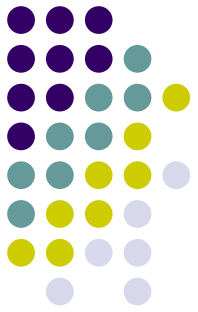
- 反引号：命令替换，可与双引号结合使用。

```
[root@node01 ~]# echo "date: `date`"
date:Wed Dec 15 15:32:04 CST 2021
```



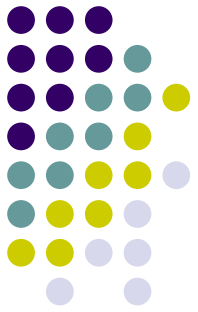
3.2 通配符

符号	含义
通配符 “*”	将与零个或多个字符匹配
通配符 “?”	与任何单个字符匹配
通配符 “[]”	与[]中的任一字符相匹配
通配符 “[!]”	与[]之外的任一字符匹配



3.3 连接符

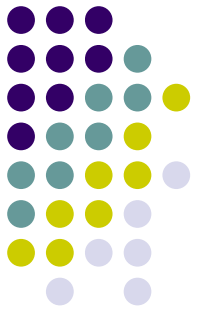
- “.” 连接符
- “&&” 连接符
- “||” 连接符



3.3.1 “;” 连接符

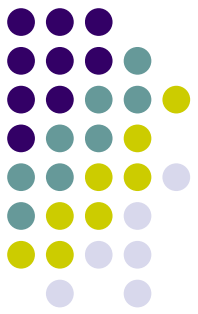
使用 “;” 运算符，命令会按照先后次序依次执行。
比如源码安装gdb包，可使用连接符连接安装命令。

```
#tar -xzvf gdb-7.11.1.tar.gz ;cd gdb-7.11.1;  
./configure;make;make install;gdb -v
```



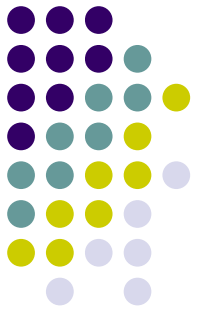
3.3.2 “&&” 连接符

使用 “&&” 运算符，其前后命令的执行遵循逻辑与关系，只有该连接符前面的命令执行成功后，它后面的命令才被执行。



3.3.3 “||” 连接符

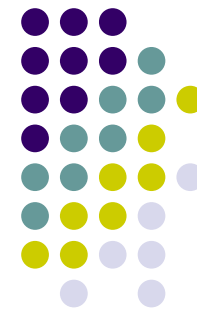
使用 “||” 运算符，其前后命令的执行遵循逻辑或关系，只有该连接符之前的命令执行失败时，才会执行后面的命令。



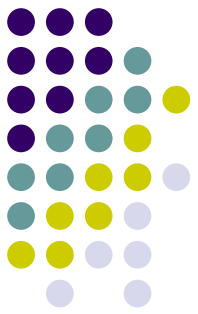
3.4 其他符号

- `;;` case分支终止符
- `:` 空命令, 相当于true
- `.` 相当于source命令
- `#` 注释

Part 4 shell脚本



- 基本元素
- 执行方式



4.1 Shell脚本的基本元素

- 第一行:

#!/开头, 指定命令解释器。

可调用任何解释器。

test.sh:

#!/bin/bash

#This is an example

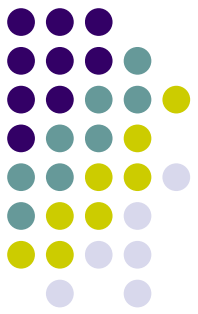
echo "Hello,Bash Shell!"

- 第二行:

#注释

- 第三行:

可执行语句。



4.2 Shell脚本的执行方式

- 1.Shell解释器运行脚本

将脚本文件作为参数传递给解释器。这种运行方式只要对文件有读取权限就可以。使用-x参数可以进行调试

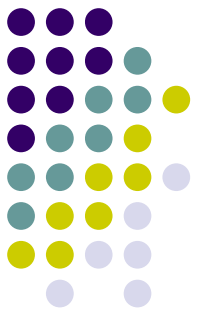
- 例:

①指定shell解释器 -
rw 权限即可.

bash test.sh

bash -x test.sh a b

sh test.sh



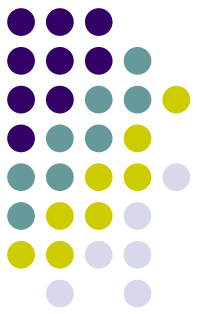
4.2 Shell脚本的执行方式

- 2.通过source运行脚本

source是Shell内部命令，它的功能是读取指定Shell程序文件，并依次执行其中的所有语句。

例： `source test.sh`

`. test.sh`



4.2 Shell脚本的执行方式

- 3. 直接运行脚本（有权限）

赋予脚本执行权限，然后直接运行脚本。

它的运行原理与第一种方式相同，都是先调用解释器创建一个子Shell，然后由解释器解释执行脚本程序。

例：

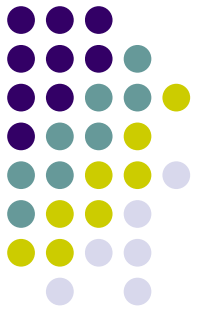
`chmod u+x test.sh`

`./test.sh`

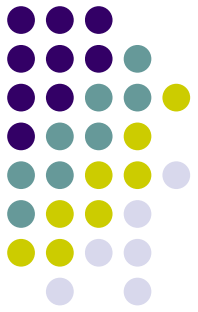
可执行权限

赋予x权限、

Part 5 shell流程控制

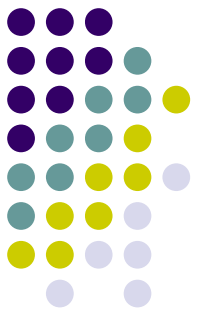


- 运算符
- 条件测试语句
- 判断结构
- 循环结构



5.1 运算符

- 算术运算符
- `expr`指令
- `let`指令
- 自增自减运算符`++`、`--`

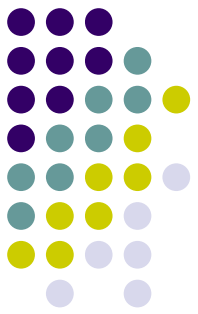


5.1 运算符

- 算术运算符：+ - * / %
- 算术表达式求值：\$[表达式] 或者 \$((表达式))
- 使用举例：

num1=\$[4+1]

num2=\$((2+\$num1*5-3))



5.1 运算符

- expr指令

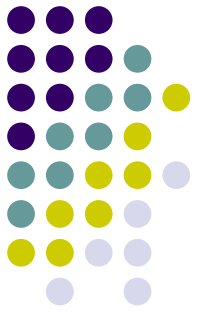
expression

求表达式变量的值，一般用于整数值，也可用于字符串
用空格隔开每个项；用反斜杠 \ 放在 shell 特定的字符前
使用举例：

expr length "this is a test"

expr 14 % 9 $\Rightarrow$ 8 *[14%9]*

expr 5 * 9



5.1 运算符

- let指令

用于执行一个或多个表达式，变量计算中不需要加上 \$ 来表示变量

使用举例：

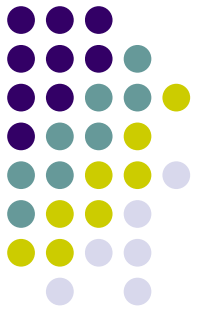
```
let a=5+4
```

```
let b=9-3
```

```
echo $a $b
```

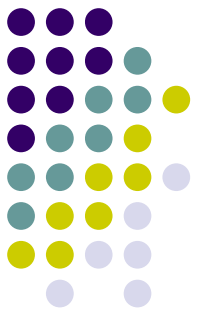
```
let a++
```

变量用美元符号



5.2 条件测试语句

- 文件测试
- 整数测试
- 字符串测试
- 逻辑运算
- 命令返回值：为0表示真、非0表示假 \$?



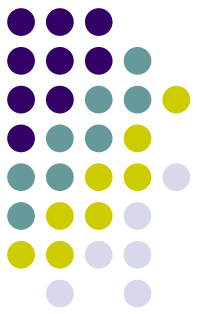
5.2 条件测试语句

- 条件测试语句

- test

语法格式: **test**空格<测试表达式>

例: **test -f file && echo true || echo false**



5.2 条件测试语句

- 条件测试语句

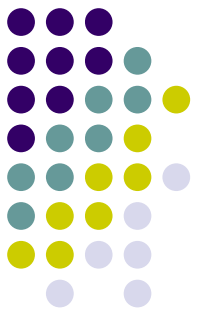
- [] 或 [[]]

语法格式: [<测试表达式>]

注意: 方括号前后都要留空格

例: [-f file] && echo true || echo false

✓ 空格



5.2 条件测试语句

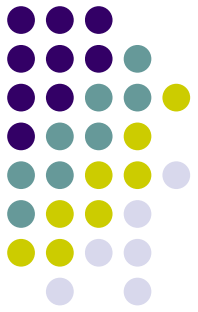
- 条件测试语句

- `[[ ]]` 支持通配符和正则表达式

语法格式: `[[ <测试表达式> ]]`

注意: 方括号前后都要留空格

例: `[[$name = [Tt]??]] && echo true ||
echo false`

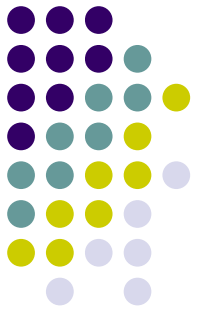


5.2 条件测试语句

- 条件测试语句

- 文件测试操作符

操作符	说明	举例
-b file	检测文件是否是块设备文件	[-b \$file]
-c file	检测文件是否是字符设备文件	[-c \$file]
-d file	检测文件是否是目录	[-d \$file]
-f file	检测文件是否是普通文件	[-f \$file]
-L file	检测文件是否是链接文件	[-L \$file]
-p file	检测文件是否是有名管道	[-p \$file]
-r file	检测文件是否可读	[-r \$file]
-w file	检测文件是否可写	[-w \$file]
-x file	检测文件是否可执行	[-x \$file]
-s file	检测文件是否为空（文件大小是否大于0）	[-s \$file]
-e file	检测文件（包括目录）是否存在	[-e \$file]



5.2 条件测试语句

```
touch shellfile
```

```
ll shellfile
```

```
[ -f shellfile ]
```

```
echo "File shellfile is a regular file?:$?"
```

```
[ -e shellfile1 ]
```

```
echo "File shellfile1 exists?:$?"
```

```
[ -d shellfile ]
```

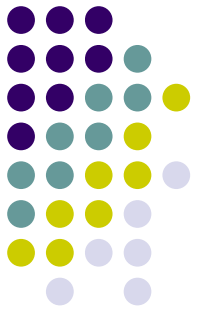
```
echo "File shellfile is a directory?:$?"
```

```
[ -s shellfile ]
```

```
echo "File shellfile is not empty?:$?"
```

```
[ -w shellfile ]
```

```
echo "File shellfile can write?:$?"
```

5.2 条件测试语句

- 条件测试语句

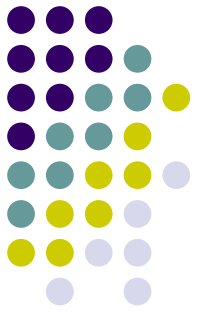
- 整数测试

整数测试通常用于数值之间的运算

语法格式：[整数1 操作符 整数2]

例：\$i -eq 0

equal

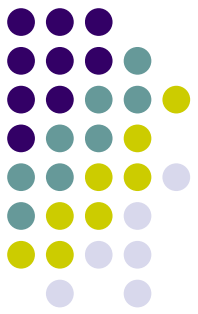


5.2 条件测试语句

- 条件测试语句

➤ 整数测试操作符，中括号和双中括号[]、 [[]]

操作符	含义
-eq	等于 (Equal)
-ne	不等于 (Not Equal)
-gt	大于 (Greater Than)
-lt	小于 (Lesser Than)
-le	小于或等于 (Lesser or Equal)
-ge	大于或等于 (Greater or Equal)



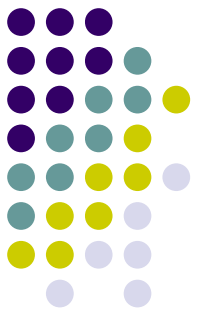
5.2 条件测试语句

- 条件测试语句

- 整数测试

也可以使用C语言中的关系运算符，用于比较两个变量的大小，比较的结果是一个布尔值，即true或false。注意要用**双小括号(())**。

符号	含义
==	等于 (Equal)
!=	不等于 (Not Equal)
>	大于 (Greater Than)
<	小于 (Lesser Than)
<=	小于或等于 (Lesser or Equal)
>=	大于或等于 (Greater or Equal)



5.2 条件测试语句

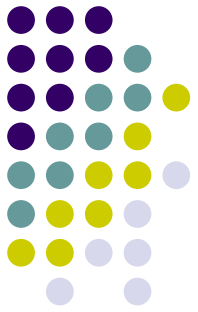
- 条件测试语句

- 字符串测试

字符串测试操作符的作用包括比较字符串是否相同、测试字符串的长度是否为0。

语法格式：[字符串1 = 字符串2]、[字符串1 != 字符串2]或[-z 字符串]

例： `$user !=root`



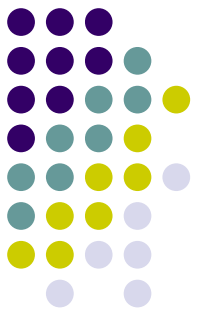
5.2 条件测试语句

- 条件测试语句

- 字符串测试

符号	含义
-z	判断字符串长度是否为0
-n	判断字符串长度是否为非0
!=	判断两个字符串是否不相等
==	判断两个字符串是否相等

不为0 即为非0



5.2 条件测试语句

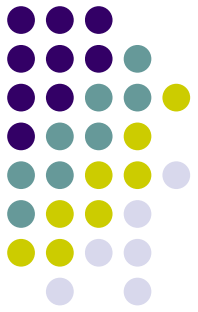
- 条件测试语句

- 逻辑运算符测试

语法格式：[表达式1] 操作符 [表达式2]

或：命令1 操作符 命令2

例：[\$x -eq 1 -a -n \$name]

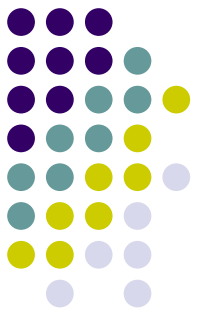


5.2 条件测试语句

- 条件测试语句

- 逻辑运算符测试

运算符	含义
-a或 &&	判断操作符两边均为真，结果为真，否则为假，" 逻辑与"
-o或	判断操作符两边一边为真，结果为真，否则为假，" 逻辑或"
!	判断操作符两边均为假，结果为真，否则为假，" 逻辑否"

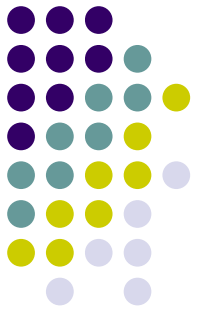


5.2 条件测试语句—逻辑测试举例

```
x=1;name=Tom;[ $x -eq 1 -a -n $name ]
```

```
n=5;m=7;[ $n -gt $m ] False && echo T True || echo F
```

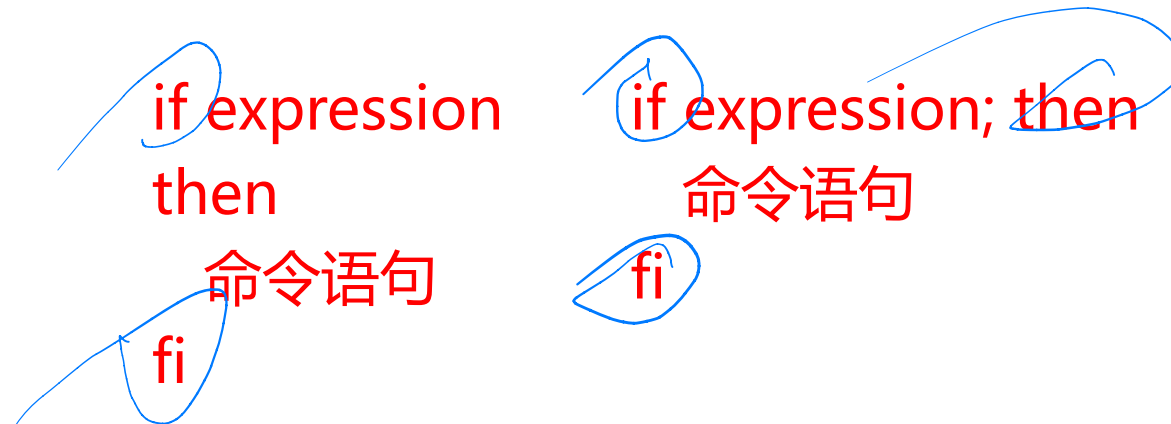
```
n=5;m=7;(($n < $m ))&& echo T || echo F
```

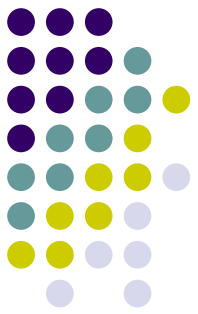



5.3 判断结构

1、if结构

if语句是最简单的判断结构，它对条件进行判断，如果判断结果为真。





5.3 判断结构

2、if/else结构

if/else结构指如果满足条件则执行then后面的操作，
如果不满足条件，则执行else后面的操作。

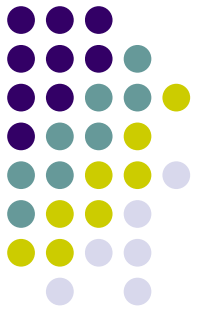
if expression; then

命令语句1

else

命令语句2

fi



5.3 判断结构

3、if/elif/else结构

if/elif/else结构可以进行多种情况的判断。

if expression1; then

命令语句1

elif expression2; then

命令语句2

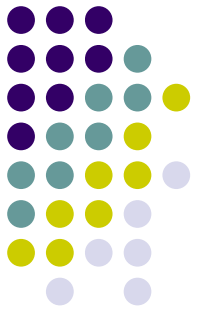
.....

else

命令语句n

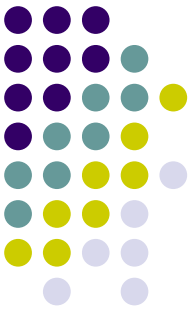
fi

if语句举例1

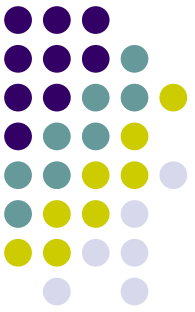


```
if [[ $answer = [Yy] || $answer = [Mm]aybe ]]
then
echo "Glad to hear it."
fi
```

if语句举例2

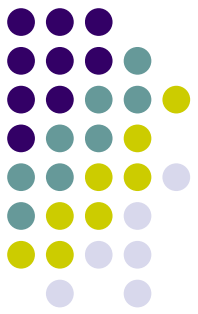


```
if [ $# -eq 1 ]  
then  
    if who|grep $1>/dev/null  
    then echo "$1 is active"  
    else echo "$1 is not active"  
    fi  
else  
    echo "Usage:$0:<username>"  
    exit  
fi
```



if语句举例3

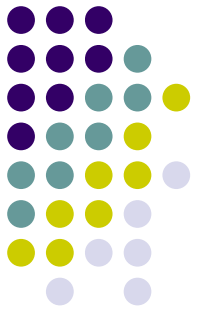
```
#!/bin/bash
echo "please input score(0-100):"
read score
if [ $score -lt 0 -o $score -gt 100 ]; then
    echo error score
elif [ $score -ge 90 ]; then
    echo A
elif [ $score -ge 80 ]; then
    echo B
elif [ $score -ge 70 ]; then
    echo C
else
    echo D
fi
```



5.3 判断结构

- if/else使用总结

- if语句必须以if开头，以fi结尾
- elif可以有任意个（0个或多个）
- else最多只有一个（0个或1个）
- 命令语句如果为空，使用冒号，返回退出状态0
- expression如果为命令，以最后一个命令的退出状态为条件值

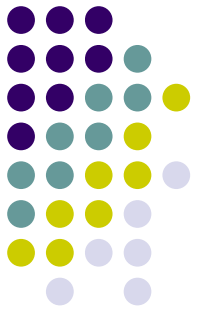


5.3 判断结构

4、case结构

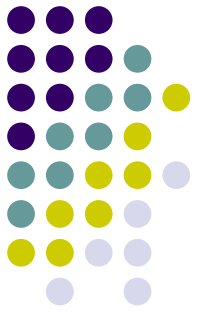
case结构也用于进行判断。

```
case "$var" in
var1)
    命令语句1;;
.....
*)
    命令语句n;;
esac
```

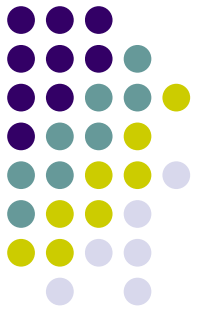
case语句举例1

```
echo "What is your preferred scripting language?"  
read -p "1)bash 2)perl 3)python 4)ruby:" lang  
case $lang in  
  1)echo "you select bash" ;;  
  2)echo "you select perl" ;;  
  3)echo "you select python" ;;  
  4)echo "you select ruby" ;;  
  *)echo "I do not know" ;;  
esac
```



case语句举例2

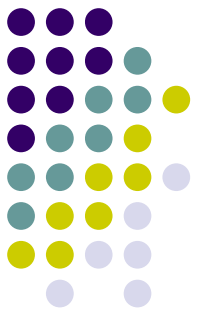
```
echo "Which is your preferred scripting language?"  
read -p "1)bash 2)perl 3)python 4)ruby:" lang  
case $lang in  
  [Bb])echo "you select bash" ;;  
  [Pp])echo "you select perl or python" ;;  
  [Rr])echo "you select ruby" ;;  
  *)echo "I do not know" ;;  
esac
```



5.3 判断结构

- case使用总结

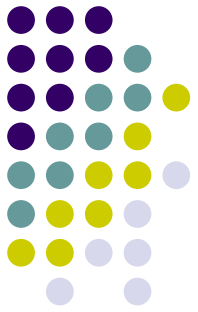
- 按顺序匹配，匹配成功则执行该分支的所有命令
- 没有匹配的模式，则执行默认值*)后的命令，可以没有
- 每个分支最后必须以双分号结束



5.4 循环结构

1、for循环

for循环是Shell编程中最常用的循环，它通常从一个列表中读取元素赋予变量，然后执行循环体中的命令语句，执行完毕之后再次从列表中读取元素赋予变量，再次执行循环体中的命令语句，如此循环直到列表中元素被读取完毕。

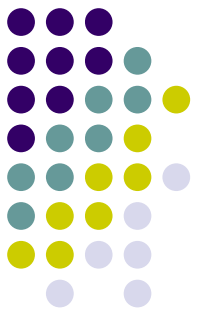


5.4 循环结构

1、for循环1

```
for var in {list}  
do  
    命令语句  
done
```

list可以是命令替换、变量替换、字符串和文件名，
可以包含通配符，用空格间隔



5.4 循环结构

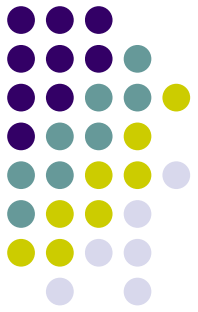
1、for循环

如果用for循环遍历一个列表，则有三种写法：

(1) `for var in 1 2 3 4 5 6` #1~6列表

(2) `for var in {1 ..6}` #1~6列表简写形式

(3) `for var in {1 ..6..2}` #以2为步长，读取1~6列表



5.4 循环结构

1、for循环2

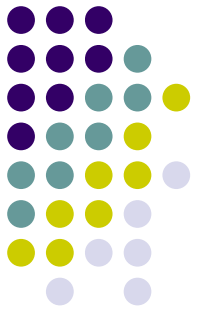
for var

do

命令执行语句

done

shell自动将命令行输入的所有参数依次组成列表



5.4 循环结构

1、for循环3

for((expression1; expression2; expression3))

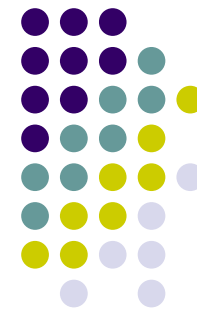
do

命令执行语句

done

C语言语法的for循环

for语句举例1

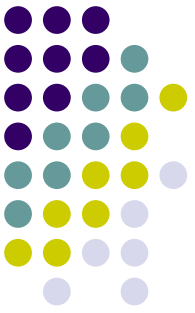


使用字符串列表

```
for x in centos ubuntu gentoo opensuse  
do  
    echo $x  
done
```

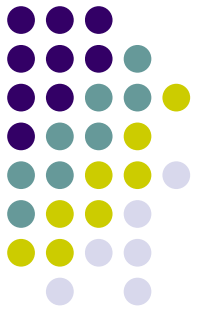
每个取出的字符串

for语句举例2



使用命令组合列表

```
for x in ls "du -sh" "df -h"  
do  
    echo "=== $x ==="  
    eval $x  
done
```



for语句举例3

使用当前目录下的文件名和目录名列表

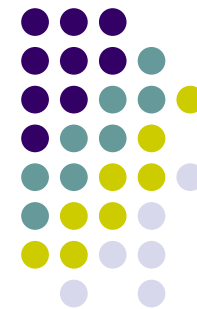
```
for fname in *
```

```
do
```

```
    echo "`ls -l $fname`"
```

```
done
```

for语句举例4



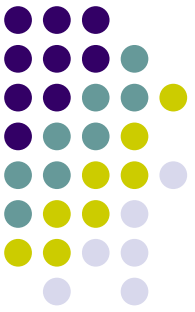
使用命令替换列表

```
for filename in `ls`  
do
```

等于 *

```
    echo $filename
```

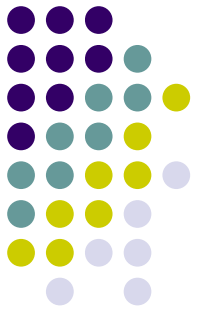
```
done
```



for语句举例5

使用命令行输入参数列表

```
i=1
for day
do
    echo -n "Positional parameter $((i++)):$day"
    case $day in
        [Mm]on|[Tt]ue|[Ww]ed|[Tt]hu|[Ff]ri)
            echo "(weekday)"
            ;;
        [Ss]at|[Ss]un)
            echo "(weekend)"
            ;;
        *)    echo "Invalid weekday"
            ;;
    esac
done
```



for语句举例6

批量添加用户

```
for x in {1..10}
```

```
do
```

```
useradd user${x}
```

```
echo "centos"|passwd --stdin user${x}
```

```
chage -d 0 user${x}
```

```
done
```

设置密码

标准输入流

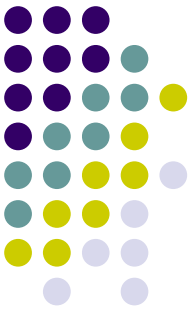
管道
将 echo 命令输出

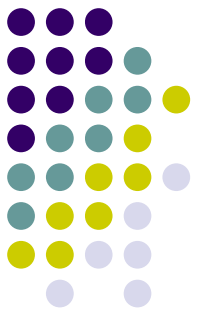
⇒ 密码已 centos.

for语句举例7

批量添加用户

```
for x in {1..50}
do
  if ((x<10))
  then name="user0"${x}
  else name="user"${x}
  fi
  useradd ${name}
done
```



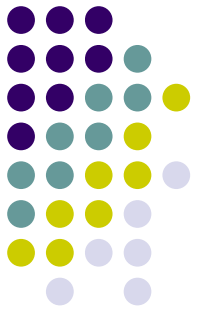


5.4 循环结构

2、while循环

while循环要简单一些，它只有一个判断条件，条件为真则执行循环体中的命令语句，条件为假则退出循环。

```
while expression  
do  
    命令语句  
done
```

while语句举例1

```
num=1
```

```
while (( $num <= 5 ))
```

```
do
```

```
    echo $num
```

```
    let num++
```

```
done
```

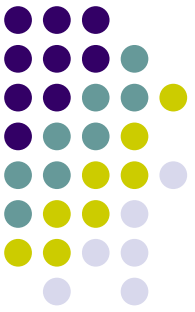
在let命令中，使变量不用加\$

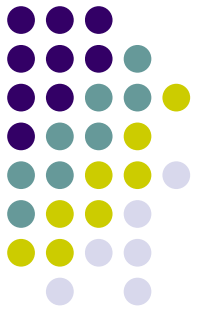
while语句举例2

while通过read读取文件内容

```
cat /etc/resolv.conf | while read line  
do  
    echo $line  
done
```

管道





while语句举例3

while循环读取命令行参数

```
echo "The count of arguments is $#"
```

→ 参数的个数、

```
echo "arguments list:"
```

```
while [ $# -ne 0 ]
```

参数不为0

```
do
```

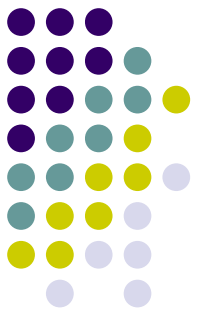
```
    echo $1
```

```
    shift
```

移动参数

```
done
```

↓
变成 \$2 ?



5.4 循环结构

3、until循环

until循环的格式与while循环类似，但在判断循环条件时，只有循环条件为假才会去执行循环体，如果循环条件为真，则退出循环。

```
until expression  
do  
    命令语句  
done
```

until语句举例1

```
#!/bin/bash
```

```
num=1
```

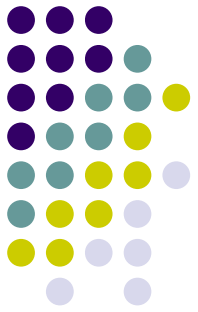
```
until [ $num -gt 5 ]
```

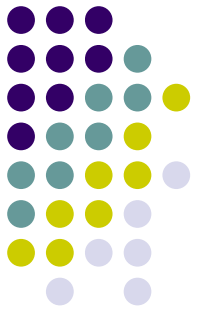
```
do
```

```
    echo $num
```

```
    let num++
```

```
done
```





until语句举例2

测试IP地址是否ping通，连通则ssh登录

```
#!/bin/bash
```

```
read -p "Enter IP address:" addr
```

```
echo $addr
```

```
until ping -c 1 $addr > /dev/null
```

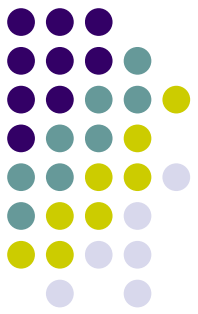
Handwritten notes in blue ink:
- Above "until": 循环条件 (Loop condition)
- Above "ping -c 1 \$addr": 判断是否连通 (Check if connected)
- Above "> /dev/null": 静默输出 (Silent output)

```
do
```

```
    sleep 60
```

```
done
```

```
ssh $addr
```



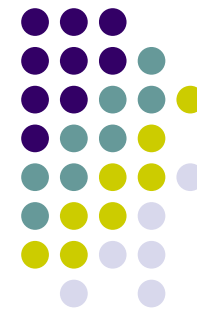
5.5 break与continue

在循环过程中，如果达到某一个条件需要退出循环，就需要使用循环控制符跳出循环，Shell中提供了两个循环控制符：break和continue。

break：退出循环

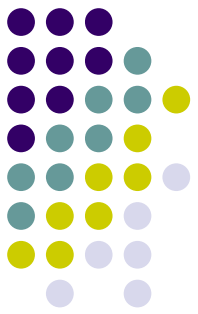
continue：结束本次循环

5.6 exit与sleep



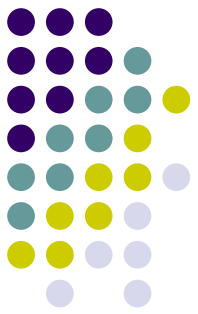
exit: 退出脚本或当前进程

sleep: 指定休眠时间



5.7 select循环

select命令是一种菜单扩展的循环方式。当程序运行到select语句时，会自动将列表中的所有元素生成成为可用1、2、3依次类推的选择列表，并等待用户输入。



5.7 select循环

语法格式：

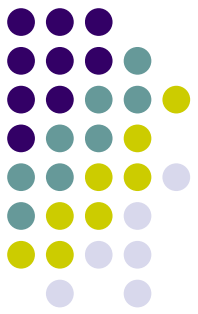
select Menu in (list)

do

command

done

当用户输入值并回车后，select可根据用户输入的值判断并执行后续的命令。如果用户不做输入，直接回车的话，select再次生成列等待输入。



select语句举例1

```
#!/bin/bash
```

```
PS3="What is your preferred scripting language? "
```

```
select s in bash perl python php ruby quit
```

```
do
```

```
    case $s in
```

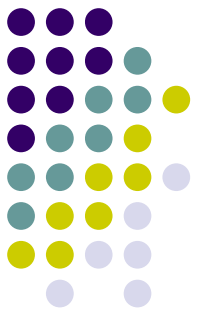
```
        bash|perl|python|php|ruby) echo "You select $s";;
```

```
        quit) exit;;
```

```
        *) echo "You select error,reply!";;
```

```
    esac
```

```
done
```



select语句举例2

PS3="Select a program you want to execute: "

TOPLIST="python telnet htop atop nettop jnettop iftop ftop
iotop mytop"

select prog in \$TOPLIST

do

case \$REPLY in

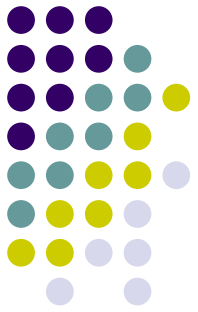
1|2|3|4|5|6|7|8|9|10)

rpm -q \$prog >/dev/null &&echo \$prog ||echo "\$prog is not
installed";;

*) exit;;

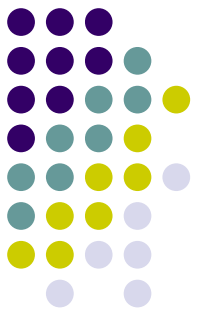
esac

done



Part 6 函数

- shell函数语法
- shell函数调用
- shell函数参数
- shell函数返回值

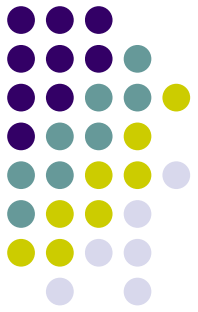


6.1 shell函数语法

语法格式:

```
[ function ] 函数名 [()]  
{  
    commands;  
    [return int;]  
}
```

```
test()  
{  
    echo "test function";  
}
```

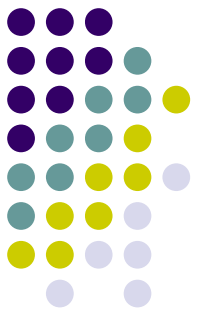


6.2 shell函数调用

语法格式:

函数名

函数名 参数1 参数2

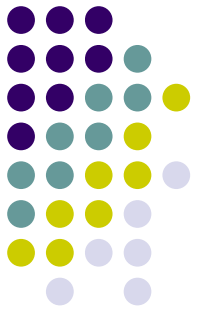


6.2 shell函数调用

函数存储

函数和调用程序保存在同一个文件：函数定义在前，调用在后

函数和调用程序分别存放在不同文件，函数文件需要先source命令执行

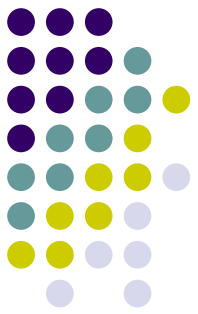


6.2 shell函数调用

函数显示

`declare -F` 显示shell可见函数名

`declare -f` 显示shell函数定义

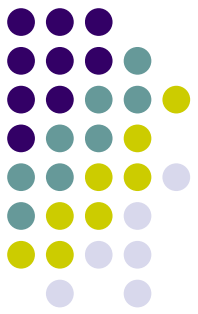


6.3 shell函数参数

调用函数时可以向其传递参数。在函数体内部，通过 `$n` 的形式来获取参数的值

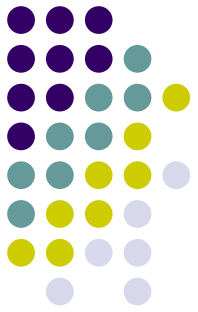
`$0`在主程序和函数中都表示脚本名

函数调用结束后，`$n`、`$*`将被重置为调用函数之前



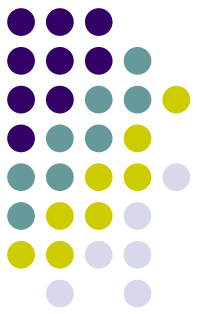
6.3 shell函数参数

```
echo "===Print positional parameters in main :"  
echo "$0: $*"  
pp1(){  
    echo 'f1--Print $* parameters in fun1:'  
    echo "$0: $*"  
}  
pp2(){  
    echo 'f2--Print $* parameters in fun2:'  
    echo "$0:$*"  
    pp1 1st 2nd 3rd 4th 5th 6th 7th 8th 9th  
    echo 'f2--Print $* parameters in fun2:'  
    echo "$0:$*"  
}  
pp1 1 2 3 4 5 6 7 8 9  
echo "===Print positional parameters in main :"  
echo "$0:$*"  
pp2 I II III IV V VI VII VIII IX
```



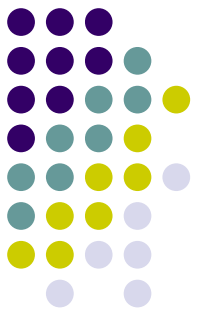
6.4 shell函数返回值

函数的返回值为最后一条命令的退出码，保存在\$?
中，return的返回值只能返回0-255之间的整数



6.4 shell函数返回值

```
max(){  
    if [[ -z $1 || -z $2 ]]  
    then  
        echo "Need 2 parameters to the function."  
        exit  
    fi  
    [ $1 -eq $2 ] &&  
    { echo "The two numbers are equal."  
      exit  
    }  
    [ $1 -gt $2 ] && return $1 || return $2  
}
```



6.4 shell函数返回值

```
read -p "Please input two integer numbers :" n1 n2
```

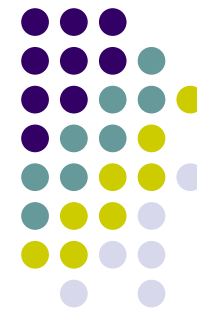
```
echo "n1=$n1,n2=$n2"
```

```
max $n1 $n2
```

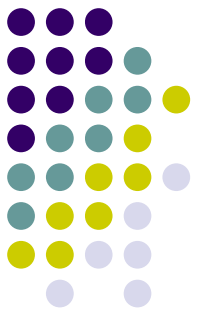
```
return_val=$?
```

```
echo "The larger of the two numbers is $return_val."
```

Part 7 正则表达式

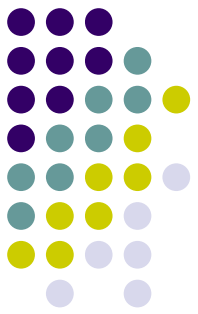


- 概念
- 元字符



7.1 正则表达式概念

- 正则表达式是对文本进行过滤（分割、匹配、查找、替换）的工具。
- 它定义了一系列的元字符，通过元字符配合其它的字符来表达出一种规则，只有符合规则的文本才能保留下来，而不符合规则的文本则被过滤掉。



7.1 正则表达式概念

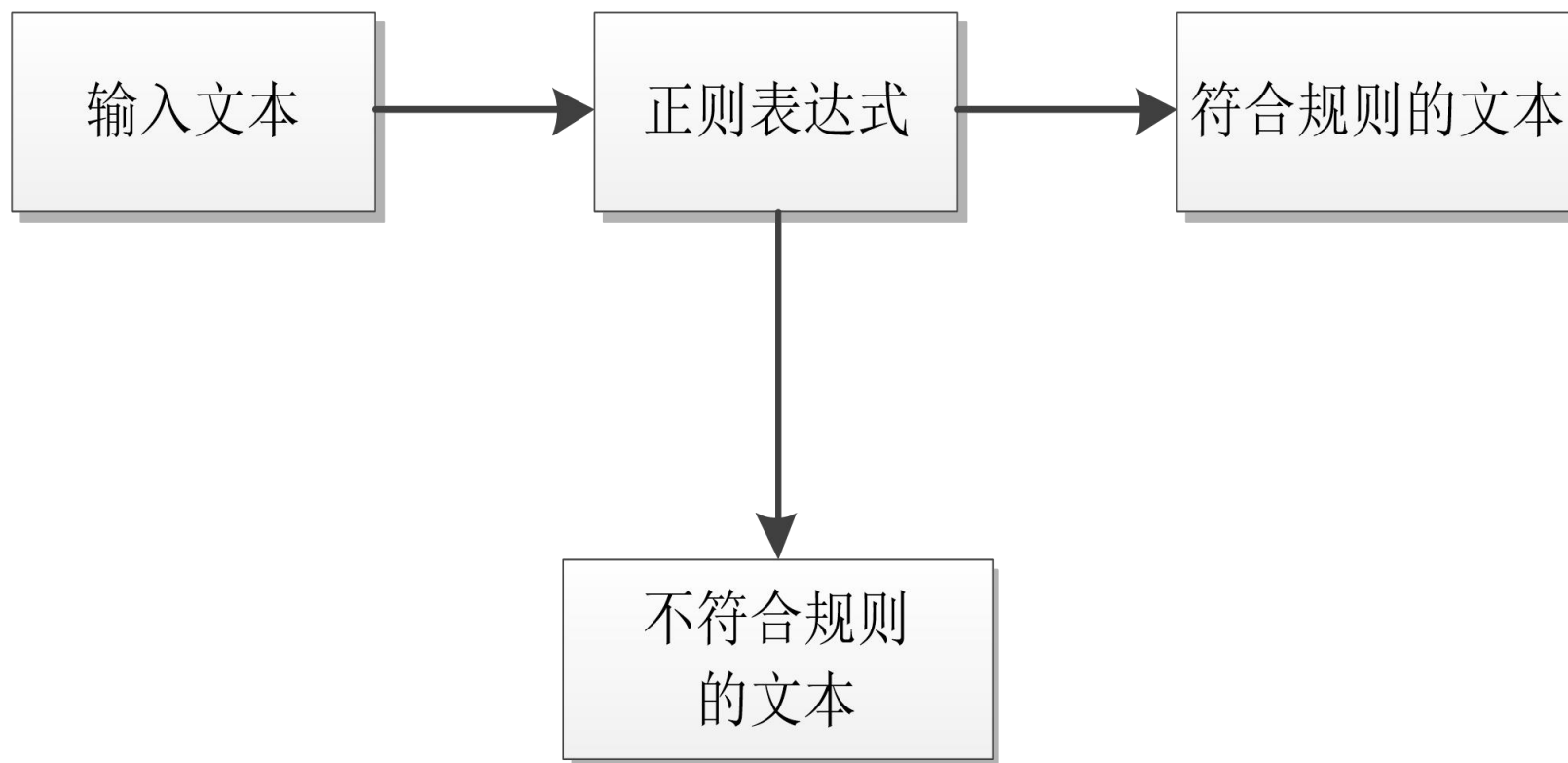
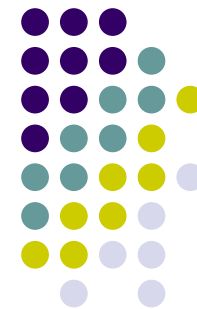
- 与通配符区别

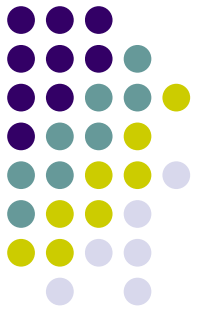
通配符：匹配文件名，完全匹配，ls,find,cp命令

正则表达式：匹配字符串，包含匹配，

grep,awk,sed命令

7.1 正则表达式概念

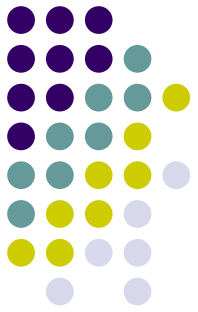




7.2 元字符

● 1. 基础元字符

- (1) 限定符 “*” : 用于匹配前面一个普通字符0次或多次重复。
- (2) 单个字符匹配符号 “.” : 匹配任意单个字符 (除却 “\n”) 。
- (3) 行首定位符 “^” : 用来匹配行首字符。
- (4) 行尾定位符 “\$” : 定位文本行的末尾 与 “^” 作用相反。
- (5) 字符集匹配符号 “[]” : 用于指定一个匹配的字符集集合。
- (6) 字符集不匹配符号 “[^]” : 不匹配字符集的任意字符。



7.2 元字符

● 1. 基础元字符

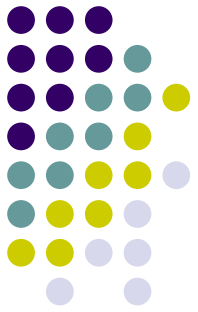
(7) 转义符 “\” : 屏蔽一个元字符的特殊意义。

(8) “{n}” : 匹配前面字符出现n次。 `hel{2}o`

(9) “{n,}” : 匹配前面字符至少出现n次。 `jo{3,}b`

(10) “{n,m}” : 匹配前面字符出现n次与m次之间 `m{2,5}o`

Do not use m{2,5}o.



7.2 元字符

● 1. 基础元字符举例

a^* : 匹配字符 a 0次或多次, 相当于所有字符, 包括空白字符

aa^* : 匹配包含 a 的字符串

$a..$: 匹配 a +任意两个字符 (非换行) 的字符串 $\rightarrow$ 匹配一个字符

$.^*$: 相当于 a^* , 匹配所有字符串

a : 匹配行首字符为 a 的字符串

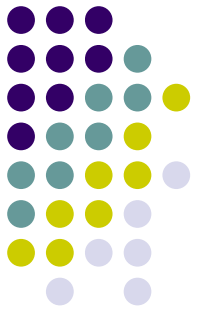
$a\$$: 匹配行尾为字符 a 的字符串

$^\$$: 匹配空白行

a

$a\$$

$^\$$ 匹配空白行



7.2 元字符

- 1. 基础元字符举例

`a[bc]d`: 匹配abd, acd

`[0-9]`: 匹配0-9中的一个数字

`^[a-z]`: 匹配行首以小写字母开头的

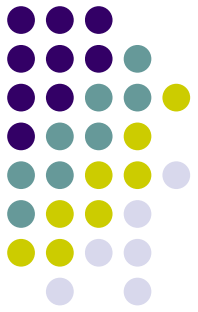
除了外面, 前面匹配.

`^[a-zA-Z]`: 匹配行首以字母开头的

`[^a]`: 匹配不包含a的字符串

`^[^a-zA-Z]`: 匹配行首不是以字母开头的

`[^a-z]`: 匹配不包含小写字母的字符串



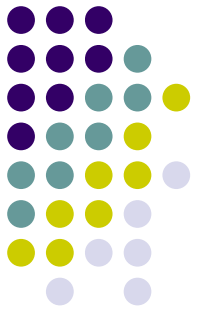
7.2 元字符

- 1. 基础元字符举例

$b\{3\}$: 匹配字符b出现3次的字符串

$b\{3,\}$: 匹配字符b至少出现3次的字符串

$ab\{3\}c$: 匹配包含abbbbc的字符串



7.2 元字符

- 1. 基础元字符举例

$\wedge a \$$

$\wedge a \{2,4\} \$$

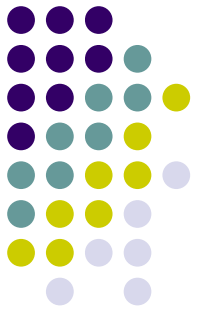
$\wedge a \{2,\} \$$

$a \{2,\} \$$

$[a-z] \{5\}$

$\wedge rc[1-9]$

$\wedge bucket \$$

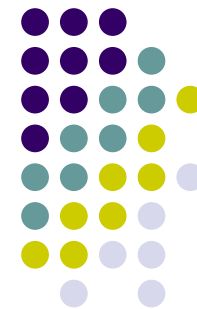


7.2 元字符

● 2. 扩展元字符

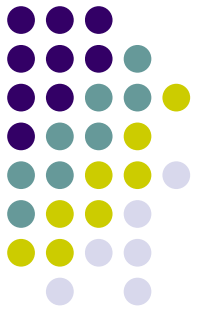
- (1) 限定符 “+”：匹配其前面的字符1次或多次。
- (2) 限定符 “?”：限定前面的字符最多出现1次（即0次或1次）。
- (3) “|” 符号：表示多个正则表达式之间的“或”关系。
表达式1 | 表达式2 | ... | 表达式n
- (4) “()” 符号：表示一组可选值的集合。“|” 与 “()” 常结合使用。

正则表达式总结



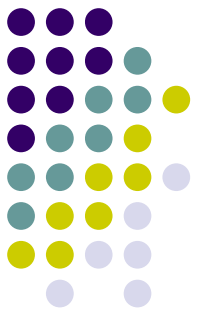
正则表达式总结





常用字符簇

- $^{[1-9][0-9]\{0,\}}\$$ 正整数
- $^{\backslash-\{0,1\}[0-9]\{1,\}}\$$ 整数
- $[0-9]\{4\}-[0-9]\{2\}-[0-9]\{2\}$ 日期
- $[0-9]\{1,3\}\backslash.[0-9]\{1,3\}\backslash.[0-9]\{1,3\}\backslash.[0-9]\{1,3\}$ IP地址
- 正则在线测试: <http://tool.rbtrees.cn/regtool/>



7.3 grep命令

在文件中全局查找。

输入流可以是标准输入流、管道和文件

ps | grep sshd

grep /bin/bash /etc/passwd

找到匹配的表达式:

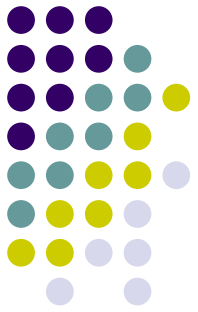
grep 返回的退出状态为 0

没找到匹配的表达式:

grep 返回的退出状态为 1

找不到指定文件:

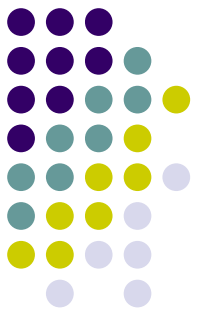
grep 返回的退出状态为 2



7.3 grep命令

grep常见选项：

- -i: 忽略字符大小写
- -q: 不输出任何信息
- -n: 显示匹配字符串所在行号
- -c: 显示成功匹配的行数
- -v: 显示不匹配的行
- --color: .bashrc或者.bash_profile文件中加入alias
grep=grep -color=auto, 生效后, grep的搜索结果自动高亮匹配。



7.3 grep命令举例

- 查找/etc/sysctl.conf文件中以“#”符号开头文本行，并显示每行的行号。

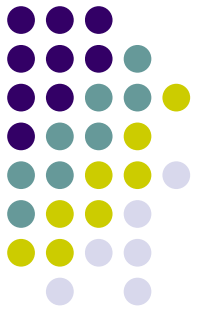
```
grep -n ^# /etc/sysctl.conf
```

- 搜索/etc/yum.conf文件中的空白行的行数。

```
grep -c ^$ /etc/yum.conf
```

- 搜索文件/etc/passwd中匹配root的字符串

```
grep root /etc/passwd
```

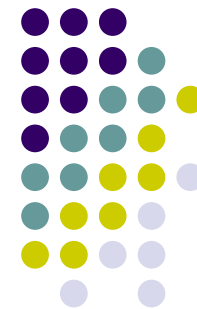


7.3 grep命令举例

- 搜索文件/etc/profile中匹配不是以#开头的行
grep -v ^# /etc/profile
不匹配
- 显示/proc/meminfo文件中以大小写s开头的行
grep ^[Ss] /proc/meminfo
- 显示/etc/passwd文件中不以/bin/bash结尾的行
grep -v /bin/bash\$ /etc/passwd
以此结尾

grep 进阶

Part 8 cut命令

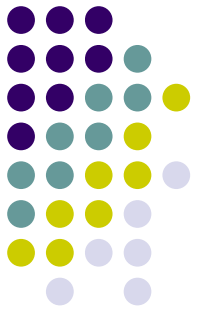


- 显示每行从开头算起 num1 到 num2 的文字
- 语法

`cut [-bn] [file]`

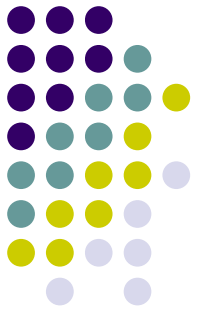
`cut [-c] [file]`

`cut [-df] [file]`



Part 8 cut命令

- 参数:
 - -b : 以字节为单位进行分割。
 - -c : 以字符为单位进行分割。
 - -d : 自定义分隔符, 默认为制表符。
 - -f : 与-d一起使用, 指定显示哪个区域。
 - -n : 取消分割多字节字符。仅和 -b 标志一起使用。



Part 8 cut命令

- 使用举例:

who|cut -b 2-4 2-4字节. my/huix... → y li

who|cut -b -3 从1-3

who|cut -b 3-,-3

从3到行尾。

cat cut.txt |cut -nb 1 不按字节切

-3, 从1到3。

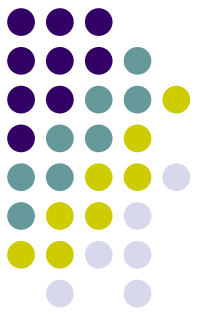
cat cut.txt |cut -c 1 按字符切

cat /etc/passwd|head -n 5|cut -d : -f 1

cat /etc/passwd|head -n 5|cut -d : -f 1,3-5

第1, 3, 4, 5列

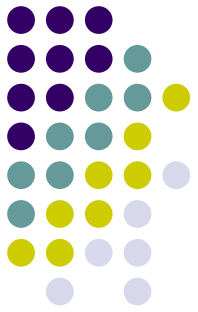
前边



Part 9 awk

awk是一种非常强大的数据处理工具，其本身是一种程序设计语言。

字符串行处理工具，用来统计、制表



Part 9 awk

- awk由两部分组成:

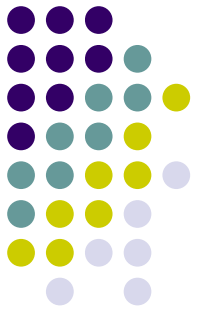
模式匹配

如果没有指定匹配模式，则默认匹配所有数据。

处理数据

匹配，则处理数据；
否则不作任何处理。
若没有指定处理方式，则输出。

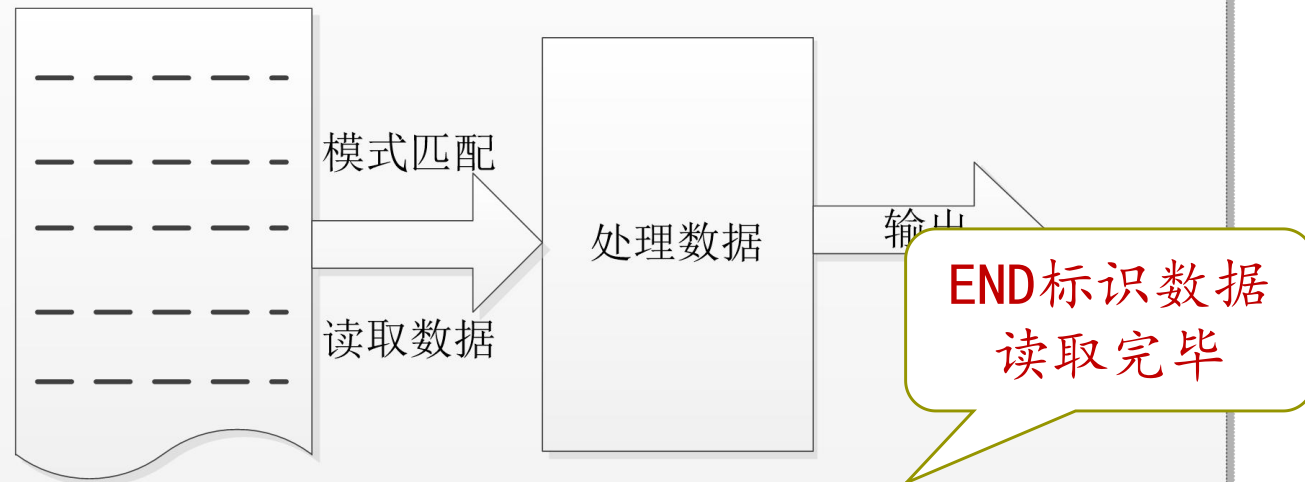
Part 9 awk



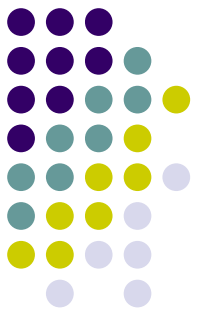
BEGIN标识开始读取数据

第一阶段：读取数据之前，由BEGIN标识

第二阶段：



第三阶段：读取数据结束，由END标识

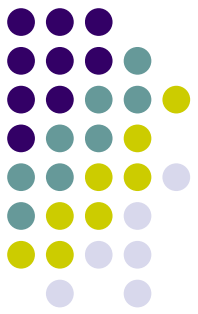


9.1 awk语法格式

awk [选项] 'BEGIN{actions} pattern {actions}END{actions}' 文件

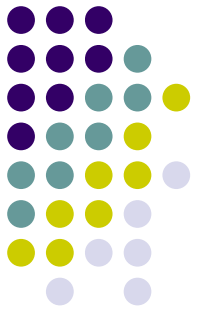
- 执行流程：

- 第一步：执行BEGIN{actions}语句块中的语句；
- 第二步：从文件或标准输入（stdin）读取一行，然后执行pattern{actions}语句块，它逐行扫描文件，从第一行至最后一行；
- 第三步：当读至输入流末尾时，执行END{actions}语句块。



9.1 awk语法格式

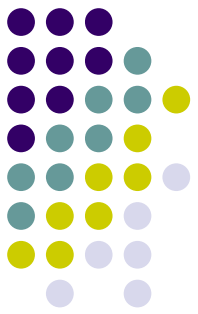
- awk [选项] ^{可以省略} 'BEGIN{actions} ^{正则执行} pattern {actions} ^{取数据} END{actions}' 文件
- **-F 选项**: 指定作为输入行的分隔符, 默认是tab键。 ^{对命令进行分隔}
 - **-v 选项**: 赋值一个用户定义的变量。
 - **-f 选项**: 从脚本文件读取awk指令。



9.1 awk语法格式

awk [选项] 'BEGIN{actions} pattern {actions}END{actions}' 文件

- ✓ pattern: 条件
一般使用关系表达式作为条件
- ✓ actions: 动作
格式化输出语句
流程控制语句

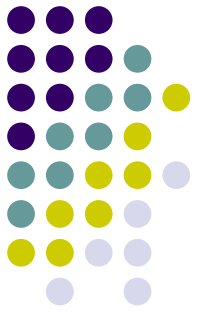


9.1 awk语法格式

匹配模式1——关系表达式

awk允许用户使用关系表达式作为匹配模式，当文本行满足关系表达式时，将会执行相应操作。

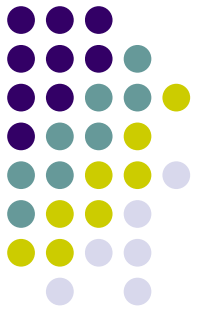
运算符	含义	运算符	含义
>	大于	==	等于
<	小于	!=	不等于
<=	小于等于	~	匹配正则表达式
>=	大于等于	!	不匹配正则表达式



9.1 awk语法格式

匹配模式2——逻辑运算符

运算符	运算	范例	结果
!	非	!a	如果a为假，则!a为真
&&	与	a&&b	如果a为真，则!a为假
	或	a b	只要a、b有一个为真， 则结果为真；二者都为 假，结果为假。

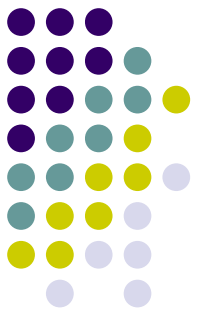


9.1 awk语法格式

匹配模式3——正则匹配模式

匹配正则表达式：

- 全行数据正则匹配//
- 全行数据正则匹配取反!// 不匹配。
- 特定数据正则匹配~// 指定字段
- 特定数据正则匹配取反!~//



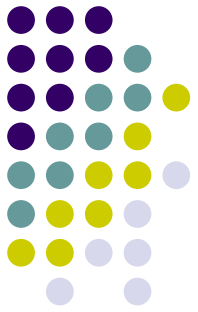
9.1 awk语法格式

匹配模式4——区间模式

匹配一段连续的文本行。

pattern1, pattern2

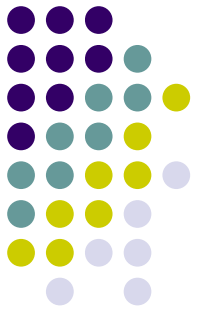
从匹配pattern1的文本行开始，到匹配pattern2的文本行结束，中间连续几个文本行都会被匹配到。



9.1 awk语法格式

● awk使用方式

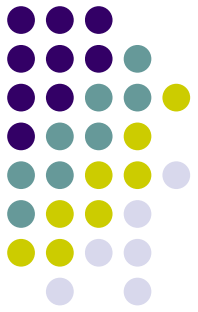
- ✓ awk 'pattern' filename
- ✓ awk '{action}' filename
- ✓ awk ^{增加}'pattern {action}' filename



9.1 awk语法格式

● awk使用举例

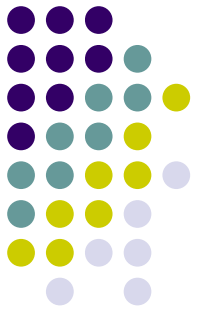
- ✓ `awk '/root/' /etc/passwd`
pattern 匹配新行 ⇒ 为每个匹配的行执行命令。
- ✓ `awk '{print}' /etc/passwd`
action 打印所有行。
- ✓ `awk '/root/ {print}' /etc/passwd`
- ✓ `awk -F: '{print}' /etc/passwd`
- ✓ `awk -F: '/root/,/mysql/{print}' /etc/passwd`
范围分解：
- ✓ `awk -F: 'BEGIN{print "User infos"}/root/{print}'`



9.2 awk常用变量

● awk常用变量

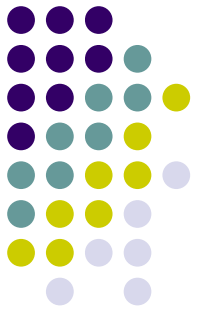
- ✓ \$0: 当前行的全部字段
- ✓ \$n: 当前行第n个字段
- ✓ NR: 行号
- ✓ NF: 当前行的字段个数
- ✓ \$NF: 最后一个字段值
- ✓ FILENAME: 当前文件名



9.2 awk常用变量

● awk常用变量举例

- ✓ `awk -F: '/root/{print $0 }' /etc/passwd`
- ✓ `awk -F: '/root/{print $1,$3 }' /etc/passwd`
- ✓ `awk -F: '/root/{print "UserName: "$1 }' /etc/passwd`
- ✓ `awk -F: '/root/{print NR,NF,$1 }' /etc/passwd`
Handwritten note: 行 总行数 分-号
- ✓ `awk -F: '/root/{print NR,NF,$1 }END{print FILENAME }' /etc/passwd`
Handwritten note: 文件名称



9.2 awk常用变量

● awk使用正则表达式

✓ ^,\$,.,*,+,?,[],[^],(),|,\

awk -F: 'BEGIN{count=0}\$1!~/^\$/ {count++} END{print count}' /etc/passwd

awk '/root/' /etc/passwd

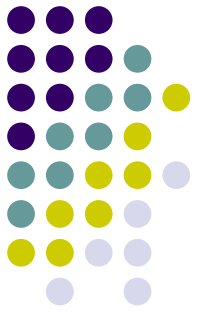
ls -l | awk '\$1~/^-/'

不匹配

每个\$|匹配到count++

匹配成功

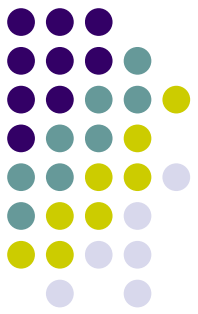
行首减号



9.3 awk流程控制

- awk流程控制

- ✓ if语句
- ✓ while
- ✓ for



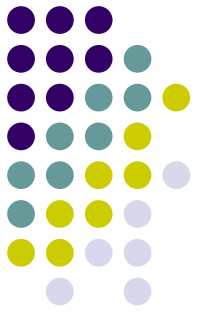
9.3 awk流程控制

● awk流程控制举例

- ✓ awk -F: '{if (\$3 < 1000) {name="system"} else {name="user"}}; {print \$1,name}' /etc/passwd
nod ' 判断用户类型.
- ✓ awk -F: '{if (\$3 == 0) {++i} else
if(\$3 < 1000) {++j} else {k++}}; END {print "admin:"i, "system:"j, "user:"
k}' /etc/passwd
统计次数.
- ✓ awk -F: '/^root/{i=1; while(i < NF) {print \$i; i++}}' /etc/passwd
遍历每一行.

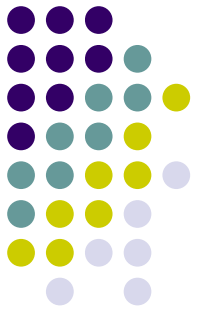
Part 10 sed

可以在脚本中实现编辑文件

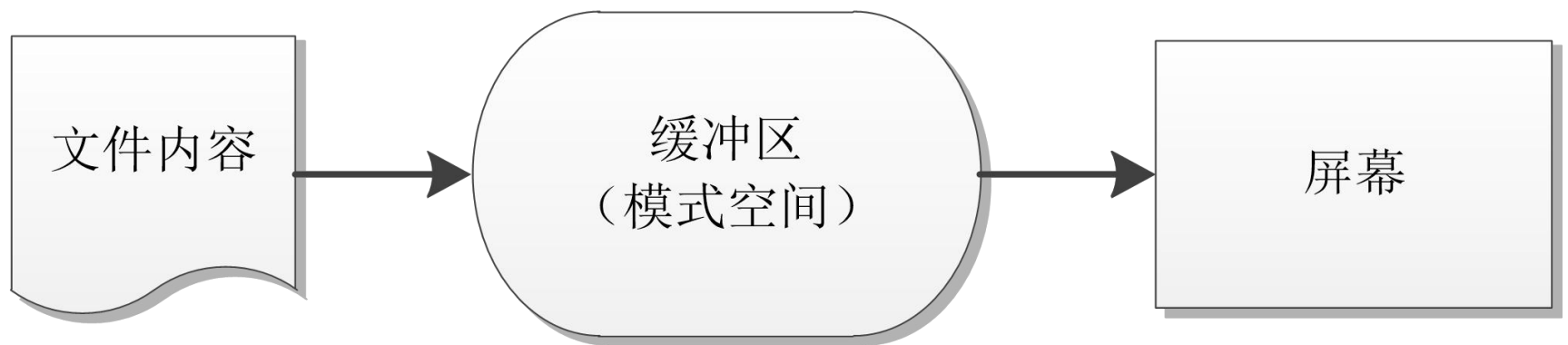


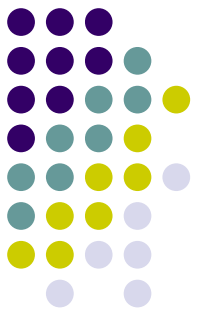
sed是Linux系统中用于文本处理的一款工具，
用于对文本进行过滤和替换，能够完善的结合正则表达式使用。

Part 10 sed



sed一次读取一行内容进行处理。
它通过文件或管道将内容读取到模式空间进行处理，
然后输出到屏幕。



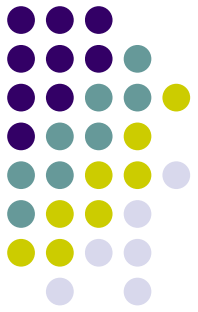


10.1 sed语法格式

语法格式: **sed [选项] 'sed编辑命令' 输入文件**

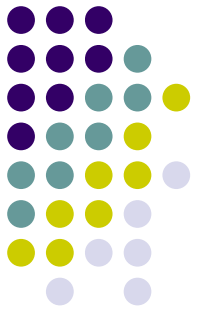
- -n选项: 关闭默认的模式空间输出。
- -e选项: 将下一个字符串解析为命令。
- -f选项: 编辑脚本内容, 表明正在使用脚本。
- -r选项: 在脚本指令中使用扩展正则表达式。
- -i选项: 直接修改原文件, 此选项要慎用。

不加-i 修改的是模式空间的内容。



10.1 sed语法格式

sed常用编辑命令	
选项	含义
p	打印匹配行
=	显示文件行号
a\	在定位行号之后追加文本信息
i\	在定位行号之前插入文本信息
d	删除定位行
c\	用新文本替换定位文本
s	使用替换模式替换相应模式
r	从一个文件中读文本
w	将文本写入到一个文件
{	在定位行执行命令组



10.2 sed行定位

- 指定行号

```
sed -n '1p' /etc/passwd
```

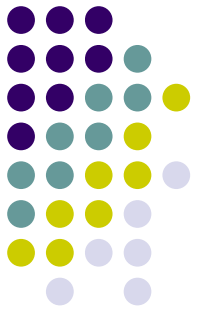
- 指定正则匹配

```
sed -n '/root/p' /etc/passwd
```

- 指定起始行

```
sed -n '1,3p' /etc/passwd
```

```
sed -n '1,/daemon/p' /etc/passwd
```

10.2 sed行定位

- 指定间隔行

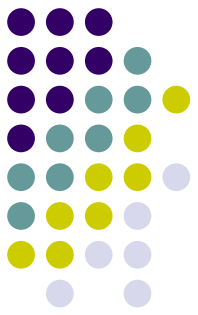
1, 4, ...

```
sed -n '1~3p' /etc/passwd
```

间隔3行

- 指定最后一行

```
sed -n '$p' /etc/passwd
```



10.3 sed操作——打印

- p命令

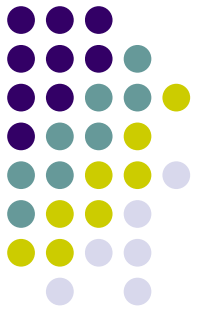
```
sed 'p' /etc/passwd
```

- =命令

```
sed '=' /etc/passwd
```

- -n选项

```
sed -n 'p' /etc/passwd
```



10.3 sed操作——追加

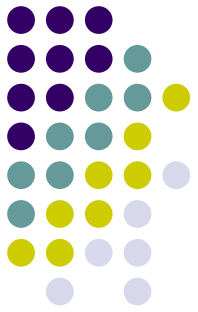
1. 追加文本

只在模式空间中追加
原文件没变，

sed '行定位a\text' 输入文件

命令实现：

```
$sed '/For rain/a\add a new line!!!' poem
```



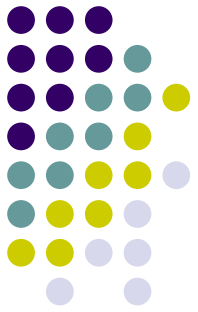
10.3 sed操作——追加

1. 追加文本

sed '行定位i\text' 输入文件

命令实现:

```
$sed '/For rain/i\add a new line!!!' poem
```



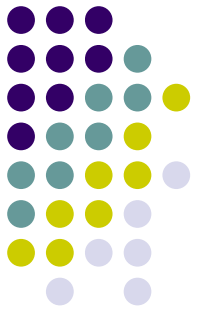
10.3 sed操作——删除

2. 删除文本

sed '行定位d' 输入文件

命令实现:

`$sed 4d poem`



10.3 sed操作——替换

3. 替换文本

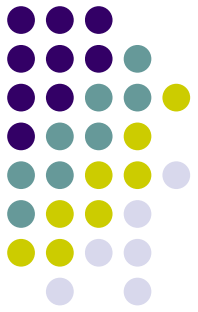
sed '行定位c\text' 输入文件

命令实现:

```
sed '3c -----' poem.txt
```

```
sed '3,5c -----' poem.txt
```

```
sed '3~5c -----' poem.txt
```



10.3 sed操作——查找替换

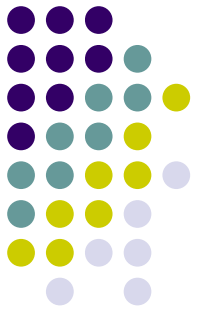
4. 查找替换

sed 's行定位/text1/text2/' 输入文件

命令实现：

```
sed 's/is/****/' poem.txt
```

```
sed 's/is/****/g' poem.txt
```



10.3 sed操作——保存文件

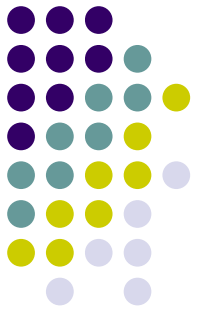
- w命令 *保存文件*
- >重定向

sed '行定位w 新文件' 输入文件

命令实现:

将第3行保存到file
sed '3w file' poem.txt

sed 's/is/****/g' poem.txt > file



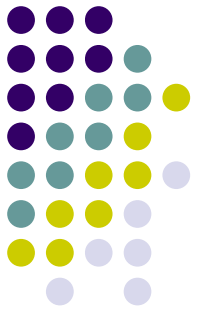
10.4 sed应用

- 禁用selinux

```
sed -i '/SELINUX/{s/permissive/disabled/}'  
/etc/selinux/config
```

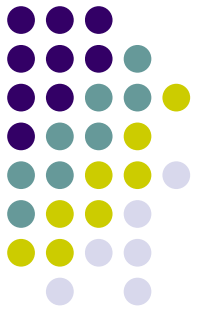
- 禁止root远程登录

```
sed -i 's/#PermitRootLogin yes/PermitRootLogin no/'  
/etc/ssh/sshd_config
```



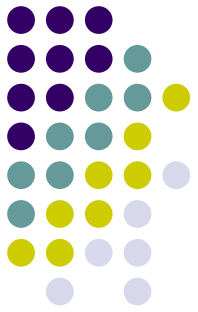
Part 11 shell脚本运维实例

- 模拟用户登录
- 监控系统运行情况
- 统计进程的状态情况



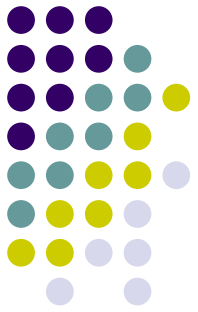
11.1 模拟用户登录

```
#!/bin/bash
echo "user:"
read user
echo "passwd:"
read passwd
if [ "$user" = "qust" ] && [ "$passwd" = "123456" ]; then
echo "SUCCESS!"
else echo "user name or passwd ERROR!"
fi
```



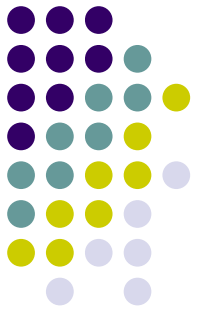
11.2 监控系统运行情况

```
#!/bin/bash
echo " (1) *****"
date;
echo "Active User:"
W
echo " (2) *****"
echo "Linux process:"
top -b | head -6
echo " (3) *****"
echo "Disk and Memory use ratio:"
df -h | xargs | awk '{print "Free/total disk:"$11/"$9}'
free -m | xargs | awk '{print "Free/total memory:" $10/"$8"MB}'
```



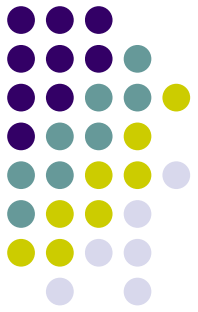
11.2 监控系统运行情况

```
echo " (4) *****"  
echo "All Processes:"  
ps auxf  
echo " (5) *****"  
echo "vmstat:"  
vmstat 1 5  
echo " (6) *****"  
echo "socket for ssh process"  
ss -pl |grep sshd
```



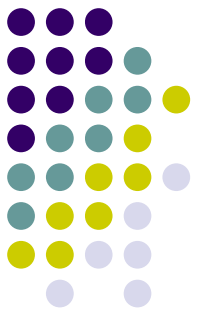
11.3 统计进程的状态情况

```
#!/bin/bash
running=0
sleeping=0
stopped=0
zombie=0
procs=0
for pid in /proc/[1-9]*
do
    procs=$((procs+1))
    stat=$(awk '{print $3}' $pid/stat)
```



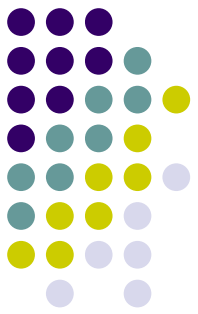
11.3 统计进程的状态情况

```
case $stat in
    R)running=$[$running+1];;
    T)stopped=$[$stopped+1];;
    S)sleeping=$[$sleeping+1];;
    Z)zombie=$[$zombie+1];;
esac
done
echo "进程统计信息如下: "
echo "总进程数为: $procs"
echo "Running 进程数为: $running"
echo "Stopped 进程数为: $stopped"
echo "Sleeping 进程数为: $sleeping"
echo "Zombie进程数为: $zombie"
```



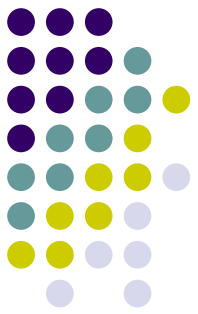
Shell案例制作1

- 完成一个监控系统资源性能的脚本案例，包括CPU的使用率与负载、磁盘和Inode利用率及IO负载、内存利用率、TCP连接状态、网络流量、分析CPU和内存占用率高的前10个进程等功能。
- 要求设计菜单和必要的输出格式控制



Shell案例制作2

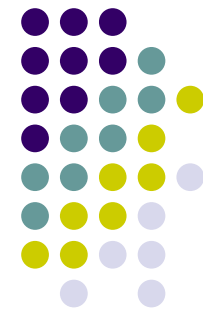
- 完成一个LNMP多机部署的案例，实现一键安装Nginx+MySQL+Php或Python环境，并实现多台机器的自动部署。
- 要求设计菜单和必要的输出格式控制



Shell案例制作3

- 完成一个Nginx日志分析案例，实现最大点击量查询、404状态查询等统计分析功能，如果日志文件过大，可以实现日志文件的定时切割（按月、天等时间切割）。
- 要求设计菜单和必要的输出格式控制

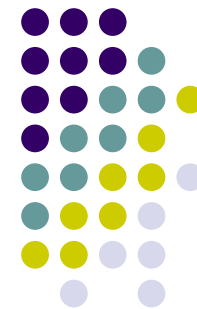
Shell案例制作4



- 完成一个MySQL系统的监控案例，实现MySQL服务状态监控、主从状态监控、数据备份等功能。
- 要求设计菜单和必要的输出格式控制

本章总结

- shell变量与符号
- shell流程控制
- shell文本处理



本章总结

- shell变量与符号
- shell流程控制
- shell文本处理

